A BACHELOR OF TECHNOLOGY PROJECT REPORT

Department of Computer Science and Engineering
(Artificial Intelligence and Machine Learning)

ML-BASED NETWORK INTRUSION DETECTION SYSTEM USING RANDOM FOREST

Machine Learning-Based Network Traffic Classification Using CIC-IDS2017

Submitted By:
Student Name: Sweti Yadav
Degree: B.Tech in CSE (AI & ML)

Institution:
Dr. Sudhir Chandra Sur Institute of Technology & Sports Complex
Kolkata, West Bengal, India

Submission Date:
23 October 2026

CERTIFICATE

This is to certify that the project report entitled **"ML-Based Network Intrusion Detection System Using Random Forest"** submitted by **Sweti Yadav** (B.Tech in Computer Science and Engineering with specialization in Artificial Intelligence and Machine Learning) is a record of bona fide work carried out under faculty supervision and guidance at **Dr. Sudhir Chandra Sur Institute of Technology & Sports Complex, Kolkata, West Bengal**, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology.

The work embodied in this report is original and has not been submitted to any other university or institution for the award of any other degree or diploma.

Faculty Supervisor
Department of CSE (AI & ML)

Head of Department
Department of CSE (AI & ML)

External Examiner / Principal
Dr. Sudhir Chandra Sur Institute of Technology & Sports Complex

DECLARATION

I hereby declare that the project report entitled **"ML-Based Network Intrusion Detection System Using Random Forest"** submitted in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning) at Dr. Sudhir Chandra Sur Institute of Technology & Sports Complex, Kolkata, is an authentic record of my own work carried out under faculty guidance.

The matter embodied in this report has not been submitted elsewhere for the award of any other degree or diploma. I have properly cited and acknowledged all external sources used in the preparation of this report.

Sweti Yadav
B.Tech CSE (AI & ML)
Date: 23 October 2026

ACKNOWLEDGEMENT

I express my profound gratitude to Dr. Sudhir Chandra Sur Institute of Technology & Sports Complex, Kolkata, for providing the necessary academic infrastructure and computational environment required to complete this project.

I am deeply indebted to my project supervisor and department faculty members for continuous encouragement, insightful technical discussions, and invaluable mentorship throughout the development of this machine learning intrusion detection pipeline.

Finally, I thank my esteemed department faculty members, laboratory staff, family, and peers whose constant support and feedback made the successful completion of this B.Tech project possible.

Sweti Yadav

ABSTRACT

Modern computer networks face unprecedented cybersecurity threats, ranging from distributed denial of service to sophisticated botnet intrusions. Traditional signature-based intrusion detection systems fail to detect zero-day or mutating malware variants. This project presents a machine learning-based network intrusion detection system (NIDS) utilizing the Random Forest classifier trained on network traffic flows derived from the benchmark CIC-IDS2017 dataset. The preprocessing pipeline handles data cleaning, binary label encoding (BENIGN versus Attack), numeric coercion, and missing/infinite value imputation. To mitigate extreme class imbalance (189,067 BENIGN records versus 1,966 Bot attack records), stratified sampling and balanced class weighting were employed. The trained model achieves high predictive reliability and is serialized via Joblib (`intrusion_model.joblib` and `feature_names.joblib`). Furthermore, an interactive Streamlit dashboard is developed to enable network administrators to upload traffic CSV files, inspect data distributions, execute inference, and download structured prediction reports.

TABLE OF CONTENTS

Chapter 1: Introduction	Page 5
Chapter 2: Literature and Technical Background	Page 6
Chapter 3: System Requirements and Architecture	Page 7
Chapter 4: Dataset and Preprocessing	Page 8
Chapter 5: Methodology and Model Development	Page 9
Chapter 6: Streamlit Application	Page 10
Chapter 7: Results and Evaluation	Page 11
Chapter 8: Limitations and Future Scope	Page 12
Conclusion & References	Page 13
Appendix: Code Excerpts & Folder Structure	Page 14

Chapter 1: Introduction

1.1 Cybersecurity Background

In the digital era, interconnected computer systems and enterprise networks form the backbone of modern infrastructure, financial institutions, and government operations. Consequently, cyber threats have evolved rapidly in sophistication, scale, and frequency. Securing these networks against unauthorized access, data exfiltration, and malicious disruptions is paramount.

1.2 Network Intrusion Detection Systems (NIDS)

Network Intrusion Detection Systems (NIDS) monitor network traffic for suspicious activity and known threats, issuing alerts when potential security breaches occur. Unlike firewalls that primarily permit or block traffic based on static port and IP rules, NIDS inspects packet payloads and flow dynamics to identify malicious behaviors.

1.3 Motivation

Traditional signature-based detection engines rely on static databases of known attack fingerprints, rendering them ineffective against zero-day exploits and polymorphic malware. Artificial Intelligence and Machine Learning (ML) offer the capability to generalize from historical traffic patterns, identifying anomalous deviations associated with intrusion attempts.

1.4 Problem Statement

Modern network traffic datasets (such as CIC-IDS2017) contain hundreds of thousands of high-dimensional records with severe class imbalance, where normal traffic heavily outnumbers malicious flows. Building an efficient machine learning classifier that accurately distinguishes benign sessions from attack vectors without generating excessive false alarms remains a critical engineering challenge.

1.5 Project Objectives

- To preprocess and clean high-dimensional network flow data from CIC-IDS2017.
- To handle missing/infinite values and encode multi-class labels into binary categories (Benign vs. Attack).
- To train and evaluate a robust Random Forest classifier incorporating balanced class weighting.
- To serialize trained model assets using Joblib for deployment.
- To develop an intuitive Streamlit web dashboard for offline CSV traffic classification and reporting.

1.6 Project Scope

This project focuses on offline analysis and classification of network flow CSV logs. It does not perform live network packet sniffing, active firewall rule modification, or automated real-time blocking.

Chapter 2: Literature and Technical Background

2.1 Signature-Based vs. Anomaly-Based Detection

Intrusion detection architectures historically fall into two paradigms:

- **Signature-Based Detection:** Matches packet bytes or byte sequences against pre-configured attack signatures. Highly accurate for known attacks but blind to novel variations.
- **Anomaly-Based Detection:** Establishes a baseline of normal network behavior and flags statistical deviations as potential intrusions. Capable of detecting zero-day attacks but susceptible to higher false positive rates.

2.2 Machine Learning in Intrusion Detection

Supervised machine learning algorithms—such as Support Vector Machines, Decision Trees, and Random Forests—have demonstrated exceptional capability in modeling complex decision boundaries across multi-dimensional network telemetry features.

2.3 Random Forest Fundamentals

Random Forest is an ensemble learning method constructed from numerous independent Decision Trees during training. Each tree outputs a class prediction, and the final classification is determined by majority vote. This ensemble approach mitigates overfitting, handles high-dimensional feature spaces effectively, and maintains robustness against noisy data.

2.4 CIC-IDS2017 Dataset Background

The CIC-IDS2017 dataset, compiled by the Canadian Institute for Cybersecurity, captures benign traffic and modern attack scenarios (including Brute Force, Heartbleed, Botnet, DoS, DDoS, Web Attacks, and Infiltration). It provides realistic network flow captures with labeled ground truth, making it a gold standard for NIDS research.

2.5 References

- Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and dataset evaluation. *ICISSP*, 108-116.
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.
- Sommer, R., & Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. *IEEE S&P*, 305-316.

Chapter 3: System Requirements and Architecture

3.1 Hardware and Software Requirements

Component	Specification / Requirement
Development Environment	Google Colab (Python 3.10+)
Core Libraries	Pandas, Scikit-learn, NumPy, Joblib, Streamlit
Hardware (Training)	Multi-core CPU / Cloud GPU (Google Colab Standard Runtime)
Deployment Platform	Localhost / Streamlit Community Cloud

3.2 System Architecture Diagram

The architecture illustrates the offline training pipeline and the interactive Streamlit dashboard inference workflow.



3.3 Module Descriptions

- **Ingestion Module:** Loads raw CSV packet capture logs using Pandas.
- **Preprocessing Module:** Encodes target labels, imputes NaN/infinite values, and handles downsampling.
- **Model Engine:** Trains the `RandomForestClassifier` with balanced class weights.
- **Dashboard Module:** Streamlit web application providing interactive visualization and CSV export.

Chapter 4: Dataset and Preprocessing

4.1 Dataset Structure and Label Distribution

The project utilized a network traffic CSV dataset associated with **CIC-IDS2017** (`Friday-WorkingHours-Morning.pcap_ISCX.csv`). Initial inspection via Pandas revealed a high-dimensional structure containing **191,033 records** and **79 columns**. The target feature (` Label`) contained the following distribution:

- **BENIGN:** 189,067 records
- **Bot:** 1,966 records

4.2 Binary Label Encoding

To frame the task as a binary classification problem, text labels were mapped to binary integers:

- `0 = Benign` (Normal network traffic)
- `1 = Non-Benign / Attack` (Botnet activity and malicious flows)

Note: The model maps all non-BENIGN labels to the Attack class; the inspected training CSV contained only BENIGN and Bot labels.

4.3 Data Cleaning and Numeric Conversion

Network flow datasets often contain missing entries or division-by-zero anomalies resulting in infinite values. The preprocessing pipeline executed the following steps:

1. Coerced all feature columns to numeric types (`float64` / `int64`).
2. Replaced infinite values (`np.inf`, `-np.inf`) with `NaN`.
3. Imputed missing values using column median/mean imputation.

4.4 Sampling Strategy and Class Imbalance

Due to memory constraints and severe class skew (over 98% benign traffic), a stratified sample of **50,000 records** was extracted for model development:

- **Benign Sample:** 49,506 records
- **Attack Sample:** 494 records

This dataset remains highly imbalanced. To prevent the classifier from ignoring the minority class, `class_weight="balanced"` was applied. The sample was split into 80% training (40,000 records) and 20% testing (10,000 records) using stratification.

Chapter 5: Methodology and Model Development

5.1 Random Forest Algorithm

Random Forest is an ensemble classifier that constructs multiple decision trees during training. For input feature vector x, each tree T_k outputs a prediction. The ensemble prediction is determined by majority vote:

$$H(x) = \text{mode}\{ h_1(x), h_2(x), ..., h_K(x) \}$$

5.2 Training Workflow and Hyperparameters

The `RandomForestClassifier` from Scikit-learn was configured with the following hyperparameters:

- `n_estimators = 100` (Number of decision trees in the forest)
- `random_state = 42` (Ensures reproducibility across random splits)
- `class_weight = "balanced"` (Automatically penalizes misclassifications of the minority attack class)
- `n_jobs = -1` (Utilizes all available CPU cores for parallelized tree construction)

5.3 Evaluation Metrics and Mathematical Formulas

Model performance is assessed using True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN):

```
Accuracy = (TP + TN) / (TP + TN + FP + FN)
Precision = TP / (TP + FP)
Recall = TP / (TP + FN)
F1-Score = 2 × (Precision × Recall) / (Precision + Recall)
```

Why Accuracy Can Be Misleading: In highly imbalanced datasets (e.g., 99% benign), a naive classifier predicting all benign achieves 99% accuracy while missing 100% of attacks. Thus, **Recall for the Attack class** and **F1-score** are critical evaluation metrics.

5.4 Model Serialization using Joblib

Upon completion of training, the model and extracted feature columns were serialized to disk using Joblib:

- `intrusion_model.joblib`: Contains the fitted Random Forest estimator.
- `feature_names.joblib`: Contains the list of 79 exact feature names to ensure alignment during inference.

Chapter 6: Streamlit Application

6.1 User Interface and Web Dashboard Overview

To make the intrusion detection model accessible to security analysts without requiring command-line execution, an interactive web application was developed using **Streamlit**. The dashboard provides a clean, responsive UI for CSV upload, automated inference, and graphical metric visualization.

6.2 CSV Upload and Dataset Preview

Users can upload network traffic CSV files (such as CIC-IDS2017 pcap extracts) directly through the file uploader widget. Once uploaded, the application loads the data into memory, displays dataset dimensions (rows and columns), and presents an interactive data preview table.

6.3 Prediction Generation and Summary Metrics

Upon clicking the prediction trigger, the app aligns the uploaded features with `feature_names.joblib`, scales/cleans the numeric entries, and queries `intrusion_model.joblib`. The dashboard then displays summary metric cards:

- Total Records Processed
- Predicted Potential Attacks
- Predicted Benign Records

6.4 Visualization and Report Download

The dashboard renders a clean classification summary chart illustrating the proportion of benign versus predicted attack traffic. Furthermore, a downloadable CSV report containing record indices and corresponding prediction labels is generated for offline logging.

> **Important Note on Dashboard Predictions:** The dashboard's predicted attack counts represent model inferences on uploaded traffic logs. They must not be interpreted as independently verified ground-truth attack counts.

Chapter 7: Results and Evaluation

7.1 Dataset Summary Table

Dataset Parameter	Training Subset	Observed Dashboard File
Source File	CIC-IDS2017 Sample	Friday-WorkingHours-Morning.pcap_ISCX.csv
Total Records	50,000 (Sampled)	191,033
Feature Count	79 Numerical Columns	79 Numerical Columns
Benign Records	49,506 (99.01%)	189,162 (Predicted Benign)
Attack Records	494 (0.99% Bot)	1,871 (Predicted Potential Attacks)

7.2 Actual Measured Classification Report

Model evaluation on the stratified test set comprising exactly 10,000 records (including 9,901 BENIGN and 99 ATTACK records) achieved an overall accuracy of **0.9992 (99.92%)**. The detailed classification report is presented below:

Class	Precision	Recall	F1-Score	Support
0 (Benign)	1.00	1.00	1.00	9,901
1 (Attack)	1.00	0.92	0.96	99
Macro Average	1.00	0.96	0.98	10,000

7.3 Confusion Matrix

The evaluation confusion matrix on the 10,000-record test set is structured as follows:

Confusion Matrix:
[True Negatives (TN) = 9,901 | False Positives (FP) = 0]
[False Negatives (FN) = 8 | True Positives (TP) = 91]

7.4 Dashboard Prediction Results

When processing the complete 191,033-record CSV file (`Friday-WorkingHours-Morning.pcap_ISCX.csv`) through the Streamlit dashboard:

- **Total Records Processed:** 191,033
- **Predicted Benign Records:** 189,162
- **Predicted Potential Attacks:** 1,871

Chapter 8: Limitations and Future Scope

8.1 Limitations of the Current System

- **Offline Batch Processing Only:** The system processes static CSV files exported from packet captures; it does not support live network card sniffing or real-time packet interception.
- **Class Imbalance Constraints:** While balanced class weighting mitigates skew, extreme class imbalance in network traffic can still lead to false positives when encountering unusual benign protocol behaviors.
- **Generalization Scope:** The model was trained specifically on CIC-IDS2017 sub-captures (containing Benign and Bot traffic in the inspected sample) and may require retraining when exposed to entirely different network topologies or zero-day attack vectors.
- **No Automated Mitigation:** The system identifies and flags potential anomalies but does not execute automated firewall blocking or active threat mitigation.

8.2 Future Scope of Work

- **Real-Time Packet Sniffing:** Integrating raw socket capture libraries (such as Scapy or PyShark) to enable live stream packet inspection.
- **Deep Learning Integration:** Exploring Recurrent Neural Networks (RNNs) and Transformers (such as Temporal Convolutional Networks) for sequential flow analysis.
- **Automated Incident Response:** Interfacing the detection engine with software-defined networking (SDN) controllers or automated firewall APIs for active mitigation.
- **Explainable AI (XAI):** Incorporating SHAP (SHapley Additive exPlanations) values in the Streamlit dashboard to explain why specific traffic flows are classified as attacks.

Conclusion

This project successfully designed, implemented, and evaluated a Machine Learning-Based Network Intrusion Detection System utilizing the Random Forest algorithm on CIC-IDS2017 network traffic data. By addressing challenges such as high dimensionality, missing values, and extreme class imbalance, the trained model achieved an exceptional accuracy of 99.92%. The serialized Joblib model and the accompanying Streamlit web application provide an effective, user-friendly interface for security analysts to upload traffic logs, inspect classification summaries, and export actionable detection reports. While operating as an offline batch analyzer rather than an active intrusion prevention system, this project demonstrates the immense potential of ensemble machine learning in modern cybersecurity analytics.

References

- [1] Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and dataset evaluation. *ICISSP*, 108-116.
- [2] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.
- [3] Sommer, R., & Paxson, V. (2010). Outside the closed world: On using machine learning for network intrusion detection. *IEEE S&P*, 305-316.
- [4] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *JMLR*, 12, 2825-2830.
- [5] McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 51-56.

Appendix: Code Excerpts and Project Folder Structure

A.1 Project Folder Structure

```
NIDS_RandomForest_Project/
│
├── data/
│   └── Friday-WorkingHours-Morning.pcap_ISCX.csv
│
├── models/
│   ├── intrusion_model.joblib
│   └── feature_names.joblib
│
├── app.py
├── train.ipynb
├── requirements.txt
└── README.md
```

A.2 Installation and Execution Instructions

To set up and run the Streamlit dashboard locally, execute the following commands in your terminal:

```
pip install -r requirements.txt
streamlit run app.py
```

A.3 Key Model Training Code Excerpt

```
from sklearn.ensemble import RandomForestClassifier
import joblib

# Initialize Random Forest with balanced class weights
rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
    class_weight="balanced",
    n_jobs=-1
)

# Fit model on training split
rf_model.fit(X_train, y_train)

# Serialize model and feature names
joblib.dump(rf_model, 'intrusion_model.joblib')
joblib.dump(X_train.columns.tolist(), 'feature_names.joblib')
```